

PLOTWIST

Piattaforma Web per il Logging, Scoring e Reviewing di Film

Autore:

Raffaele Andrei (166299)

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



Indice

1	Introduzione	2
1.1	Tecnologie Utilizzate	2
2	Organizzazione del Progetto	3
2.1	Utenti	4
2.2	Film	5
2.3	Recensioni	6
2.4	Raccomandazioni	7
3	Struttura del Database	8
4	Test	9

1. Introduzione

Plotwist è una piattaforma web sviluppata in Django 6.0 per catalogare, valutare e recensire film. Offre funzionalità di ricerca tramite filtri, gestione utenti, un sistema di watchlist, un sistema di recensioni e un sistema di raccomandazioni.

1.1 Tecnologie Utilizzate

- **Backend:** Django 6.0, Python 3.9+
- **Database:** SQLite3
- **Frontend:** Bootstrap 5, Django Crispy Forms
- **Librerie:** django-filter, Pillow, python-dateutil

2. Organizzazione del Progetto

La struttura del progetto Plotwist è organizzata in moduli Django per una gestione chiara e modulare delle funzionalità. La struttura principale del progetto è la seguente:

```
plotwist/
|
|-- apps/
|   |-- movies/           # Gestione film, attori, registi
|   |-- reviews/         # Gestione recensioni e "mi piace"
|   |-- users/            # CustomUser e permessi
|   \-- recommendations/  # Sistema di raccomandazioni
|
|-- config/               # Impostazioni di progetto
|-- media/                # Immagini caricate
|-- static/               # CSS e JavaScript
|-- templates/            # File HTML
'-- manage.py             # Entry point Django
```

Ogni modulo contiene i propri modelli, viste, moduli e template per una gestione modulare e scalabile del progetto. La struttura di ogni modello è la seguente:

fixtures/ Eventuale cartella contenente dati di esempio in formato JSON o YAML.

migrations/ Contiene i file di migrazione del database generati da Django.

static/ Contiene file statici specifici del modulo (CSS, JavaScript, immagini).

templates/ Contiene i template HTML specifici del modulo.

templatetags/ Contiene eventuali tag personalizzati per i template Django.

templates/ Ospita l'interfaccia utente in formato HTML/Django Template.

admin.py Configura l'interfaccia di amministrazione di Django per il modulo.

apps.py Configura le impostazioni del modulo.

filters.py Definisce i filtri di ricerca personalizzati.

forms.py Definisce i moduli di input per il modulo.

models.py Gestisce lo schema dei dati e le relazioni.

views.py Contiene la logica di business e il recupero dei dati.

urls.py Definisce il mapping tra indirizzi web e funzioni Python.

2.1 Utenti

Nella pagina generale ogni utente può effettuare il login come utente registrato oppure admin della piattaforma oppure registrarsi come nuovo utente normale oppure nuovo utente con i privilegi di moderatore.

Tutti gli utenti di Plotwist possono navigare il catalogo di film e cercare film tramite vari filtri. Per ogni film è disponibile una pagina di dettaglio con informazioni come titolo, regista e attori con le relative recensioni degli utenti.

Nella home si possono visualizzare le classifiche dei primi nove film con il voto più alto rispetto al mese corrente, all'anno corrente e di sempre.

La piattaforma è stata progettata per principalmente 4 tipologie di utenti, ogni tipologia di utente eredita i permessi della tipologia precedente:

Column Name	#	Data Type
<code>id</code>	1	INTEGER
<code>password</code>	2	VARCHAR
<code>last_login</code>	3	DATETIME
<code>is_superuser</code>	4	BOOL
<code>username</code>	5	VARCHAR
<code>first_name</code>	6	VARCHAR
<code>last_name</code>	7	VARCHAR
<code>is_staff</code>	8	BOOL
<code>is_active</code>	9	BOOL
<code>date_joined</code>	10	DATETIME
<code>email</code>	11	VARCHAR
<code>profile_picture</code>	12	VARCHAR

Figura 2.1: Struttura della tabella degli Utenti.

Per controllare che l'utente sia registrato o faccia parte del gruppo dei moderatori o amministratori sono stati utilizzati i mixin di Django:

```

1  LoginRequiredMixin
2
3  UserPassesTestMixin
4  def test_func(self):
5      return self.request.user.is_staff or self.request.user.
        groups.filter(name='Moderators').exists()

```

2.2 Film

I film sono il fulcro principale della piattaforma Plotwist. Ogni film ha attributi come titolo, regista, attori, generi, anno di uscita, trama e poster. Gli utenti possono cercare film tramite vari filtri come genere, anno di uscita e regista.

Ogni film ha una pagina di dettaglio dove sono visualizzate tutte le informazioni e le recensioni degli utenti. se un utente ha visto un film, può aggiungerlo alla propria watchlist per tenerne traccia.

Esiste una pagine che mostra i film aggiunti alla piattaforma dal utente moderatore loggato. Quando si clicca sopra un film, si viene reindirizzati alla pagina di dettaglio del film. Se il fim è statio aggiunto dal moderatore loggato, viene mostrato anche un pulsante per modificare o eliminare il film.

L'amministratore ha accesso alla modifica e cancellazione di tutti i film aggiunti da tutti i moderatori. Tutti gli utenti posso aggiungere i film alla propria watchlist dalla pagina di dettaglio del film.

Column Name	#	Data Type
123 id	1	INTEGER
A-Z title	2	VARCHAR
A-Z release_date	3	DATE
123 length	4	INTEGER
123 logged_by_id	5	BIGINT
123 director_id	6	BIGINT
A-Z poster	7	VARCHAR
A-Z plot	8	TEXT

Figura 2.2: Struttura della tabella dei Film.

La watchlist è stata implementata con un campo ManyToMany nella tabella Movie che associa gli utenti ai film presenti nella loro watchlist. L'aggiunta e rimozione di un film dalla watchlist avviene tramite una vista dedicata che verifica se il film è già presente nella watchlist dell'utente:

```

1 def post(self, request, pk):
2     movie = get_object_or_404(Movie, pk=pk)
3     movie.watched_by.add(request.user)
4     return redirect('movies:watchlist')
```

2.3 Recensioni

Nella pagina di dettaglio di ogni film, gli utenti registrati possono scrivere delle recensioni. Ogni recensione è associata unicamente ad un utente e ad un film.

Gli utenti possono esprimere un voto da 1 a 10 e scrivere un commento testuale. Viene visualizzato il voto medio del film calcolato in base a tutte le recensioni degli utenti. Inoltre, gli utenti possono mettere o togliere "mi piace" alle recensioni degli altri utenti per indicare che trovano utile la recensione. Ogni recensione mostra il numero totale di "mi piace" ricevuti, le recensioni con più "mi piace" vengono visualizzate in cima alla lista delle recensioni.

Una recensione può essere modificata o eliminata solo dall'utente che l'ha scritta o dall'amministratore della piattaforma. Quando una recensione viene eliminata, tutti i "mi piace" associati a quella recensione vengono eliminati automaticamente, e il voto medio del film viene aggiornato di conseguenza. Questo comporta un aggiornamento dei film nella classifica della home page se il voto medio cambia.

Column Name	#	Data Type
123 id	1	INTEGER
A-Z text	2	TEXT
123 rating	3	INTEGER
A-Z created_at	4	DATETIME
A-Z updated_at	5	DATETIME
123 movie_id	6	BIGINT
123 user_id	7	BIGINT
A-Z title	8	VARCHAR

Figura 2.3: Struttura della tabella delle Recensioni.

Per implementare la funzionalità di "mi piace" alle recensioni, è stata creata una tabella separata "Like" che associa gli utenti alle recensioni a cui hanno messo "mi piace". Per stabilire se un utente ha messo "mi piace" ad una recensione, è stato creato un tag personalizzato nei template Django.

In `apps/reviews/templatetags/reviews_extras.py`:

```

1 @register.simple_tag
2 def has_liked(review, user):
3     """Return True if the given user liked the review."""
4     if not user or not user.is_authenticated:
5         return False
6     return review.likes.filter(user=user).exists()

```

In `apps/reviews/templatetags/reviews_extras.py`:

```

1 {% has_liked review user as user_liked %}
2 {% if user_liked %}
3     <span> Hai messo like</span>
4 {% endif %}

```

2.4 Raccomandazioni

La piattaforma Plotwist utilizza un sistema di Content-Based Filtering che funziona nel seguente modo:

- **Identificare i film preferiti dell'utente:** il sistema estrae tutti i film che l'utente ha votato ≥ 7 (su 10), se non ci sono film votati ≥ 7 , l'utente non riceve raccomandazioni.
- **Estrazione delle preferenze:** Dal set di film votati ≥ 7 , il sistema estrae tre tipi di caratteristiche:
 - Generi preferiti: Tutti i generi dei film che l'utente ha votato
 - Registi preferiti: Tutti i registi dei film che l'utente ha votato
 - Attori preferiti: Tutti gli attori presenti nei film che l'utente ha votato
- **Ricerca di film simili:** Il sistema cerca film che NON sono stati ancora recensiti dall'utente e che condividono:
 - Almeno uno dei generi preferiti
 - Almeno uno dei registi preferiti
 - Almeno uno degli attori preferiti

```

1 user_reviewed_movies = Movie.objects.filter(
2     reviews__user=user
3 ).values_list('id', flat=True)
4
5 recommendations = Movie.objects.filter(
6     Q(genres__in=preferred_genres) |
7     Q(director__in=preferred_directors) |
8     Q(actors__in=preferred_actors)
9 ).exclude(
10     id__in=user_reviewed_movies
11 ).annotate(
12     avg_rating=Avg('reviews__rating')
13 ).distinct().order_by('-avg_rating')

```

3. Struttura del Database

In questo capitolo viene mostrato lo schema ER semplificato del database di Plotwist. Per gestire l'associazione multi-a-molti tra film e generi e tra film e attori, sono state create due tabelle intermedie "MovieGenre" e "MovieActor" che collegano i film ai rispettivi generi e attori. Allo stesso modo, per gestire la watchlist degli utenti, è stata creata una tabella intermedia "Watchlist" che associa gli utenti ai film presenti nella loro watchlist.

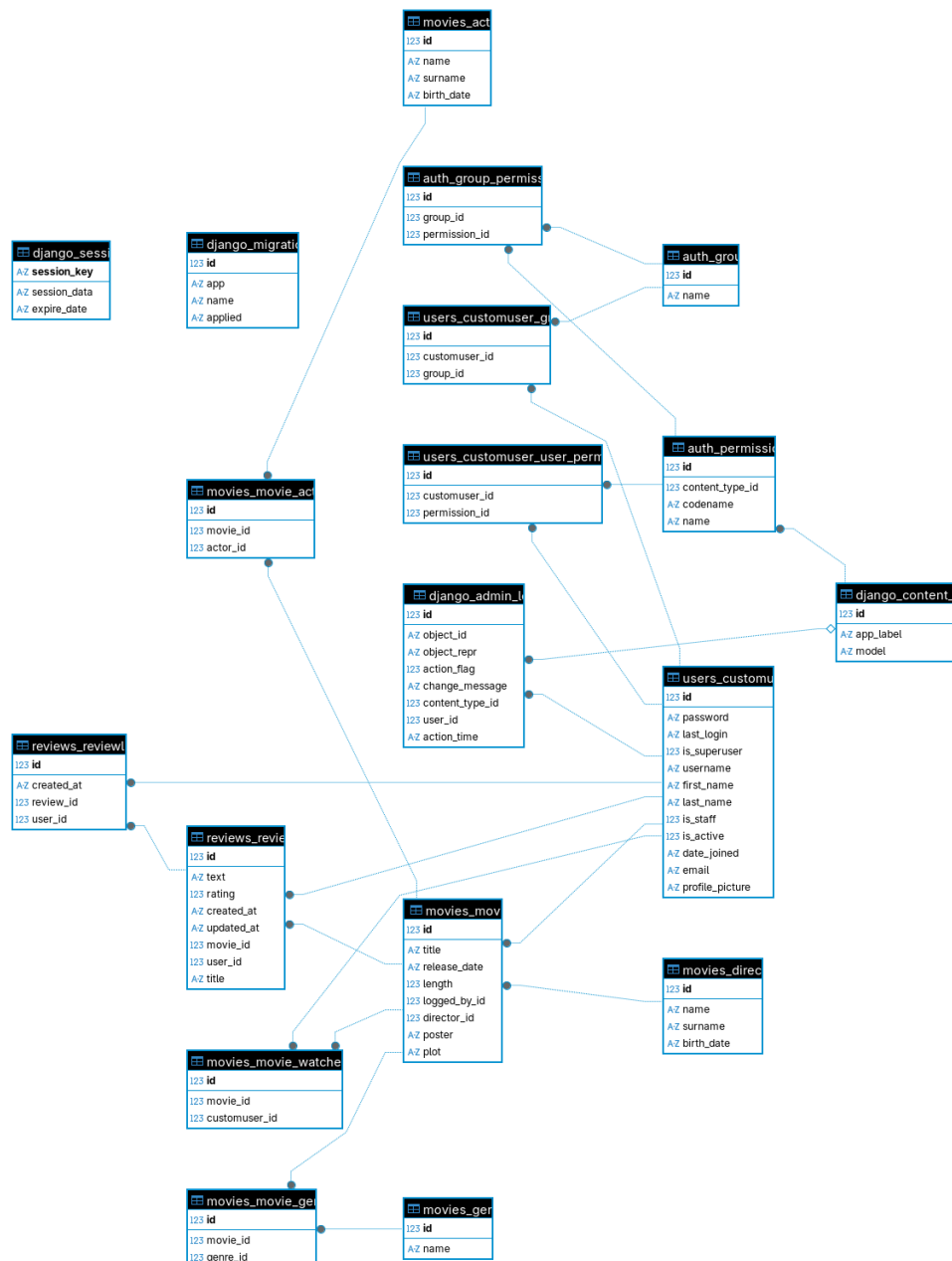


Figura 3.1: Schema ER del database.

4. Test

- test_average_rating_no_reviews** Verifica che il calcolo del voto medio di un film senza recensioni ritorni None.
- test_average_rating_single_review** Verifica che il calcolo del voto medio di un film con una singola recensione ritorni il voto corretto.
- test_average_rating_multiple_reviews** Verifica che il calcolo del voto medio di un film con più recensioni ritorni il valore corretto arrotondato a due decimali.
- test_average_rating_rounding** Verifica che il calcolo del voto medio di un film con recensioni che richiedono arrotondamento ritorni il valore corretto arrotondato a due decimali.
- test_average_rating_extreme_values** Verifica che il calcolo del voto medio di un film con recensioni ai valori estremi (1 e 10) ritorni il valore corretto arrotondato a due decimali.
- test_movie_detail_view_returns_200** Verifica che la vista di dettaglio del film ritorni uno status code 200 (OK) per un film esistente.
- test_movie_detail_view_uses_correct_template** Verifica che la vista di dettaglio del film utilizzi il template HTML corretto.
- test_movie_detail_contains_movie_data** Verifica che la vista di dettaglio del film contenga i dati corretti del film (titolo, regista, anno di uscita).
- test_movie_detail_contains_reviews** Verifica che la vista di dettaglio del film contenga le recensioni associate al film.
- test_movie_detail_context_data** Verifica che la vista di dettaglio del film fornisca i dati di contesto corretti (film, recensioni, voto medio).
- test_movie_detail_unauthenticated_user** Verifica che un utente non autenticato possa accedere alla vista di dettaglio del film.
- test_movie_detail_authenticated_not_owner** Verifica che un utente autenticato che non è il proprietario del film possa accedere alla vista di dettaglio del film.
- test_movie_detail_authenticated_owner** Verifica che un utente autenticato che è il proprietario del film possa accedere alla vista di dettaglio del film.
- test_movie_detail_average_rating_displayed** Verifica che il voto medio del film venga visualizzato correttamente nella vista di dettaglio del film.
- test_movie_detail_watchlist_button_authenticated** Verifica che il pulsante per aggiungere/rimuovere il film dalla watchlist venga visualizzato per un utente autenticato.
- test_movie_detail_404_nonexistent_movie** Verifica che la vista di dettaglio del film ritorni uno status code 404 (Not Found) per un film inesistente.

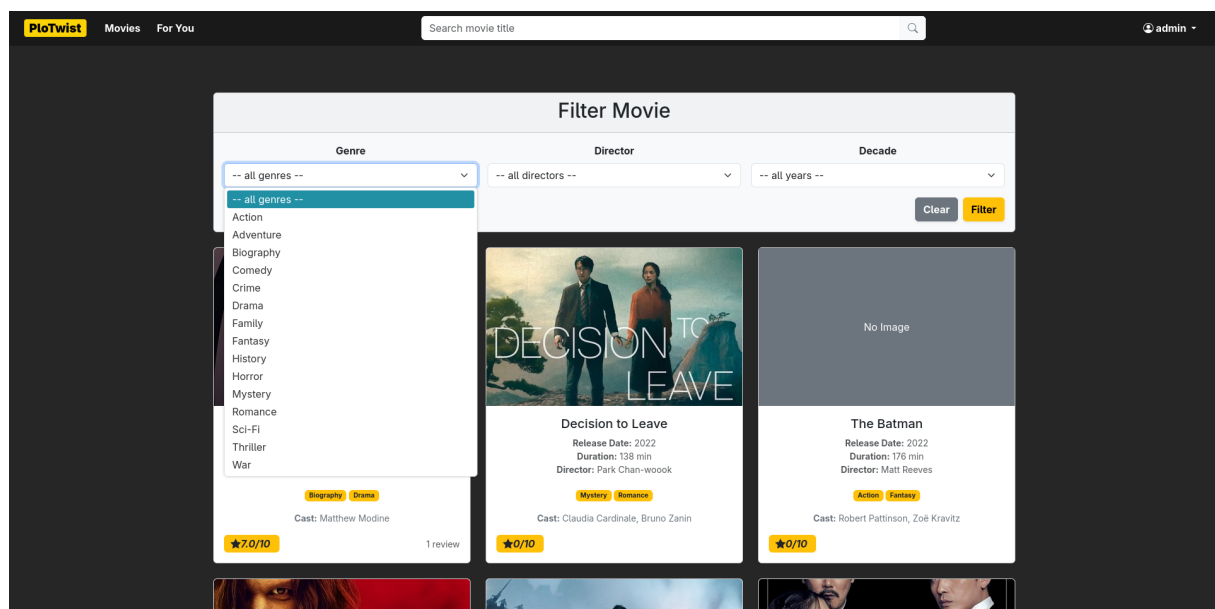


Figura 4.1: Filtro per la ricerca dei film.

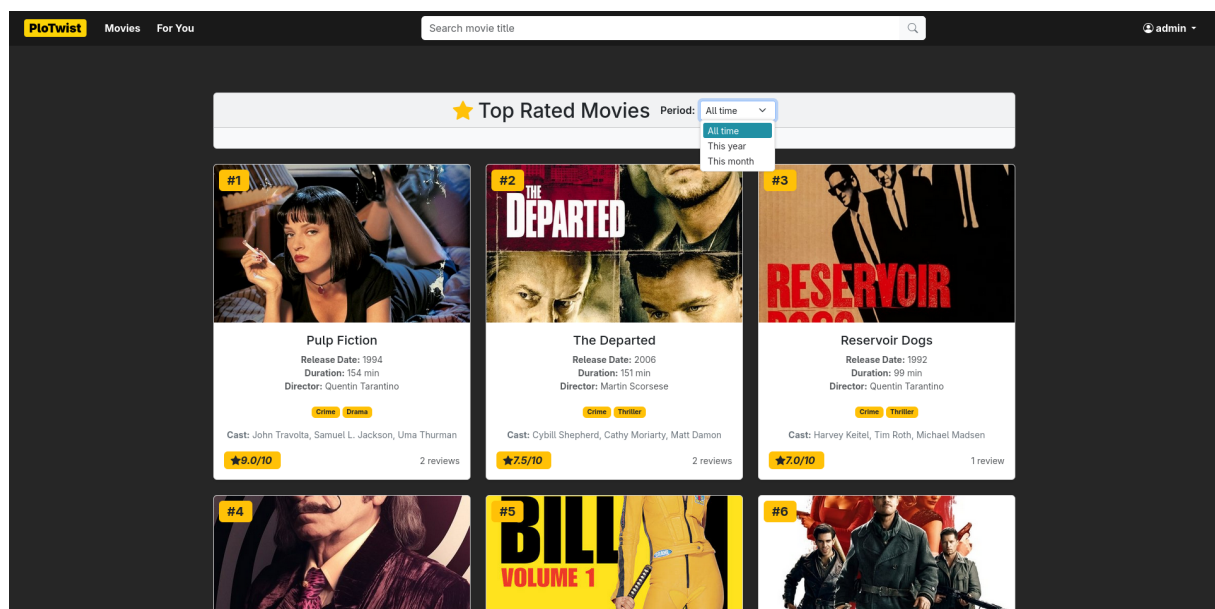


Figura 4.2: Filtro per la classifica dei film.